

SAGA: Schedule Abstraction Graphs at GPU Speed

Heng Zhao

The University of British Columbia
Vancouver, BC, Canada
hengzhao@cs.ubc.ca

Zhengan Zhao

University of Victoria
Victoria, BC, Canada
zhenganz@uvic.ca

Arpan Gujarati

The University of British Columbia
Vancouver, BC, Canada
arpanbg@cs.ubc.ca

Abstract—Accurate schedulability analysis is essential for certifying safety-critical real-time systems. Schedule Abstraction Graph (SAG), a reference framework for such analyses, explores all possible job dispatch orders while merging compatible states to limit growth. Yet, combinatorial explosion remains its critical bottleneck: current SAG algorithms are bound by single-threaded CPU traversals, severely constraining the workload sizes they can evaluate. We present SAGA, the first CPU–GPU heterogeneous framework for SAG-based analysis. By completely restructuring state expansion and merging for massively parallel execution, SAGA breaks the CPU bottleneck. We instantiate within SAGA every algorithm from the canonical SAG implementation. On an NVIDIA B200, SAGA delivers a 470× speedup at the largest workloads tractable for state-of-the-art CPU tools, with extrapolated speedups exceeding three orders of magnitude for larger systems. By reducing hours of computation to seconds, SAGA establishes a new performance standard for schedulability analyses.

I. INTRODUCTION

Schedulability analyses underpin the certification of safety-critical real-time systems. Before a system is deployed, a rigorous analysis must confirm that every task will meet its timing constraints under all possible execution scenarios. Because the true worst-case scenario is often unknown due to timing anomalies and parameter variations, simulation-based approaches cannot provide sound guarantees. Alternatively, traditional sufficient schedulability tests (such as classical response-time analysis or demand-bound methods) address this by pessimistically over-approximating the worst case, often rejecting valid task sets to maintain scalability. Conversely, exact analysis avoids this pessimism by exhaustively reasoning about the space of executions a workload can produce, but this exhaustive analysis is notoriously expensive.

Among schedulability analyses, the Schedule Abstraction Graph (SAG) [1] stands out by achieving an elegant balance between tractability and precision. Its central insight is to reframe schedulability as the systematic exploration of possible job dispatch orders: each path in the graph corresponds to one feasible scheduling decision sequence, and each node abstracts the system states reachable along that prefix. A sound merge operation then collapses semantically equivalent states, keeping the exploration tractable. This paradigm has proven remarkably extensible. Over the past decade, the SAG family has grown from non-preemptive uniprocessor scheduling [1] to global multiprocessor scheduling [2, 3], limited-preemptive parallel DAG tasks [4], event-driven workloads with self-suspensions [5], and time-aware shapers in TSN networks [6], catalyzing a robust ecosystem of analytical extensions and

supporting tools. For the system models it covers, the SAG is a natural reference point against which other analyses of those same models are measured.

This breadth, however, has come at a real cost: every variant in the SAG family inherits the same single-threaded, sequential exploration of a state graph that grows combinatorially with the number of jobs and the expressiveness of the workload model¹. While SAG implementations scale to workloads well beyond the reach of exact timed-automata encodings [9, 10], the community has openly noted that scalability remains a fundamental limitation, particularly under release jitter and execution-time variation [11]. Published analyses of more expressive variants can take minutes to hours, and do not scale to highly parallel DAG tasks or large core counts [4, 5]. The community’s response has been entirely *algorithmic* [11, 12]: partial-order reduction, lazy abstraction, sharper state representations, and more aggressive merging. These efforts have produced real gains, but they all operate within the same sequential execution model, ultimately bounded by single-core performance. Despite these algorithmic strides, breaking the sequential bottleneck through new computational paradigms remains a critical and unresolved challenge in the field.

This is the gap SAGA closes. Modern GPUs ship with tens of thousands of compute cores and are standard hardware on the servers where schedulability analysis is actually run. Yet, during analysis, these powerful parallel resources sit idle while a single CPU core does all the work. SAGA shifts the focus from purely algorithmic improvements to algorithm–hardware co-design by fundamentally restructuring the SAG computational paradigm for CPU–GPU heterogeneous systems.

¹The SAG family is canonically maintained in the SAG-org repository [7]. Its documentation lists four published algorithms, all of which we instantiate: non-preemptive uniprocessor scheduling [1], non-preemptive moldable gang scheduling [3], limited-preemptive global parallel DAG scheduling [4], and limited-preemptive self-suspending scheduling with event-driven delays [5]. Unless otherwise noted, references to the SAG family, the SAG lineage, or SAG algorithms in this paper denote exactly these results. However, the SAG-org repository and the SAG literature are not limited to these. For example, the SAG-org implementation supports combinations of workload models assumed by the four algorithms [1, 3–5], v3.3.1 supports mutual exclusions and conditional DAG tasks [8] as well, and a recent work on TSN time-aware shapers [6] builds on SAG but is not part of the SAG-org codebase.

While the reference implementation includes a CPU thread-parallel build backed by Intel TBB, only the ECRTS 2019 evaluation by Nasri et al. [4] used it; subsequent publications report single-threaded numbers. A source-level review surfaced concurrency defects that make its outputs non-reproducible across runs, so we compare against the sequential build. That a working parallel implementation has not yet landed even in the canonical codebase reinforces the broader point: parallelizing SAG correctly is non-trivial.

SAGA makes four contributions. (1) We design the first CPU–GPU heterogeneous framework for accelerating SAG-style schedulability analyses, parallelizing the expansion and merging of system states on the GPU while keeping the CPU productively engaged. (2) We introduce a set of GPU-aware optimizations that further reduce the work the analysis needs to do, without sacrificing its precision. (3) We demonstrate that SAGA generalizes across the SAG family by instantiating every algorithm documented by the canonical implementation [7], spanning non-preemptive uniprocessor, moldable gang, limited-preemptive parallel DAG, and event-driven self-suspending workloads [1, 3–5]. (4) We evaluate SAGA against Srinivasan et al.’s SAG-based analysis of limited-preemptive self-suspending tasks with event-driven delays [5] — the most computationally demanding member of the SAG lineage. SAGA achieves measured speedups of more than 470× over the corresponding state-of-the-art CPU tools; on larger workloads where existing analyses become intractable, SAGA yields its largest gains, with conservatively extrapolated speedups exceeding three orders of magnitude.

II. SAG AND THE CASE FOR HARDWARE ACCELERATION

We first review the computational structure shared by all SAG variants, and then argue why its cost motivates hardware acceleration. An event-driven delay (EDD) workload example, illustrated in Fig. 1, accompanies the review.

Consider a finite set J of jobs executing on m identical cores under a global job-level fixed-priority policy.² Each job j has a release time within $[r_j^{\min}, r_j^{\max}]$ (the bounds capture release jitter), an execution time within $[C_j^{\min}, C_j^{\max}]$, an absolute deadline d_j , and a priority p_j , with smaller p_j meaning higher priority. The SAG algorithms we instantiate differ in the workloads they model, and hence in how a job is formalized.

In the uniprocessor analysis [1], a job is just this tuple. The DAG analysis [4] adds a predecessor set $\text{pred}(j)$: j becomes ready only once every job in $\text{pred}(j)$ completes. The EDD analysis [5] further labels each precedence edge with a delay interval $[sus^{\min}, sus^{\max}]$, bounding the time between a predecessor’s completion and the successor’s readiness, modeling, e.g., a wait on an accelerator, on an I/O response, or on a synchronization event. The moldable-gang analysis [3] instead lets a job occupy several cores at once, with admissible core counts and per-count execution-time bounds. The literature treats these models separately, but the reference implementation does not, i.e., v3.3.1 [7] supports them in combination, and supports further models that we do not instantiate, among them mutual exclusion and conditional DAG tasks [8].

An *execution scenario* γ fixes one concrete value for every interval parameter; Γ denotes the set of all scenarios. Job set J is schedulable iff every job meets its deadline in every $\gamma \in \Gamma$.

²A finite job set is obtained directly, or from any task model with a predictable arrival pattern by enumerating the jobs released over a safe observation window, e.g., one hyperperiod for synchronous periodic tasks. Sporadic arrivals, which require reasoning over an infinite family of arrival sequences, lie outside the SAG model.

The analysis constructs a directed acyclic graph $G = (V, E)$ rooted at state v_0 . Each edge dispatches one job, so a path

$$\pi = v_0 \xrightarrow{j_{a_1}} v_1 \xrightarrow{j_{a_2}} \dots \xrightarrow{j_{a_i}} v_i$$

corresponds to the scheduler dispatching jobs $j_{a_1}, \dots, j_{a_i}$ in that order. Fix a scenario $\gamma \in \Gamma$: its concrete times determine the scheduler’s decisions completely, so γ follows exactly one path through the graph. A state v represents the scenarios whose paths pass through v . Layer $\mathcal{L}_i \subseteq V$ is the set of states at depth i , i.e., with i jobs dispatched. Every scenario eventually dispatches every job, one at a time, so its path descends through the layers one by one and visits exactly one state of each layer. The states of $\mathcal{L}_i$ hence collectively represent all of Γ [5]. The analysis constructs $\mathcal{L}_i$ completely before $\mathcal{L}_{i+1}$.

Every state records the set D of dispatched jobs and the availability vector $A = \langle A_1, \dots, A_m \rangle$ indicating when cores become available. Availability is tracked without naming cores: $A_k = [A_k^{\min}, A_k^{\max}]$ bounds when the system has k cores free simultaneously (i.e., possibly from $A_k^{\min}$ but certainly by $A_k^{\max}$), so A_1 bounds when the next core frees up, whichever physical core that is. For instance, an availability vector $A = \langle [0, 0], [1, 2] \rangle$ does not say core 1 is free at 0 and core 2 in $[1, 2]$; it says some core is certainly free from time 0, and both are free by some point in $[1, 2]$. Beyond D and A , each SAG variant stores the auxiliary timing information its eligibility and merge rules require. In the EDD analysis, this is the set F of finish-time intervals of dispatched jobs that still have undispatched successors (which acts as the input to those successors’ readiness) and the set χ of jobs certainly executing (used to tighten the availability bounds of successor states); the DAG and gang variants store similar information [3, 4].

Each layer is produced by two phases. The *expansion* phase visits every state of the current layer and emits one successor state for each job that could be dispatched next from that state. The *merging* phase then (at the cost of some precision) replaces groups of sufficiently similar states among those successors by a single state that abstracts all of them, which is what limits how fast a layer can grow. The two phases alternate until every job has been dispatched. We describe each in turn.

For each state v on the current layer $\mathcal{L}_i$, the expansion phase determines its *candidate* jobs that may be dispatched next in at least one scenario of v . Candidacy of a job c is checked in two stages. First, c must pass a *structural test*: it must be undispatched and, in the variants with precedence constraints, its predecessors must be dispatched. Second, a variant-specific, priority-aware *timing test* bounds the interval $[s_c^{\min}, s_c^{\max}]$ in which c could start as the next dispatch, and rejects c if the interval is empty. The jobs that pass both tests form the *candidate set* R of v . Intuitively, $s_c^{\min}$ is the earliest moment at which c could be ready with a core free, and $s_c^{\max}$ the latest moment at which the scheduler could still pick c [5]. That latest moment turns on the jobs that pass the structural test at v : c is no longer the scheduler’s choice once one of higher priority is *certainly* ready, i.e., ready in every scenario

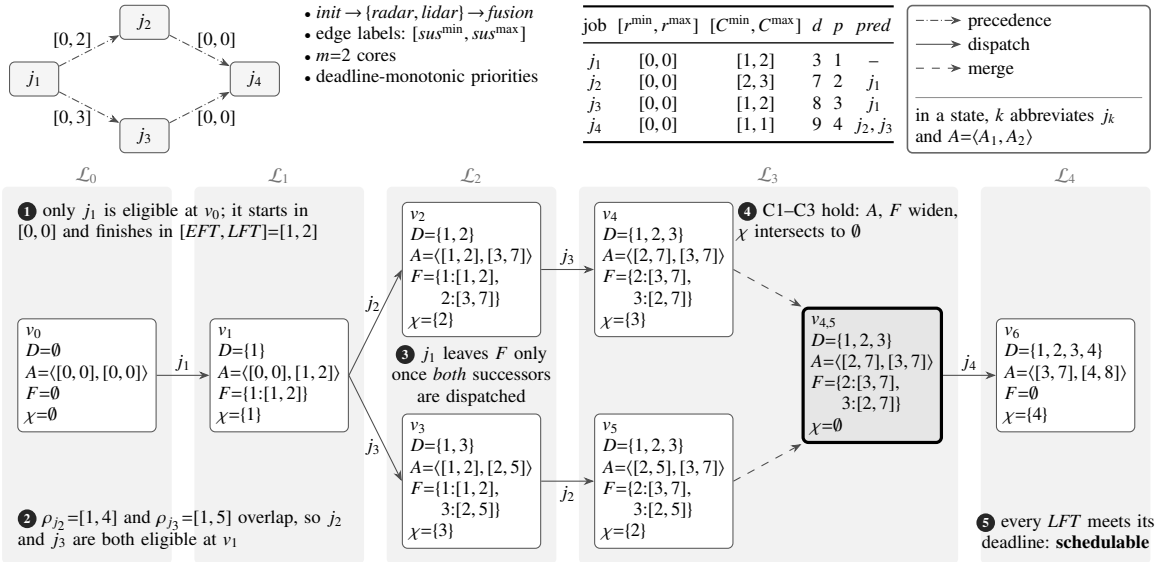


Fig. 1: SAG construction for an EDD workload on $m = 2$ cores: an object-detection task whose initialization segment j_1 feeds radar and lidar processing segments j_2 and j_3 on separate accelerators, with results fused by j_4 (adapted from [5]). Each precedence edge carries a delay bound $[sus^{min}, sus^{max}]$. **1** At v_0 , only j_1 can be scheduled; it starts at 0 and finishes in $[EFT, LFT] = [1, 2]$; this finish interval enters F because j_1 has undispatched successors. **2** At v_1 , both j_2 and j_3 are candidates. Job j 's readiness interval ρ_j combines its predecessor's finish interval in F with the edge delay: $\rho_{j_2} = [1+0, 2+2] = [1, 4]$ and $\rho_{j_3} = [1+0, 2+3] = [1, 5]$. So either can go first, and the lower-priority j_3 precedes j_2 exactly in scenarios where its delay expires earlier. Note $s_{j_3} \in [1, 3]$: once j_2 is certainly ready at 4, the scheduler would dispatch j_2 , so j_3 -first requires j_3 to start strictly earlier. **3** In $\mathcal{L}_2$ – $\mathcal{L}_3$ the remaining job (j_3 or j_2) follows; j_1 stays in F until both its successors are dispatched, and is then retired, so $|F|$ grows and shrinks as the analysis advances. **4** v_4 and v_5 satisfy C1–C3: equal D , and all availability and per-job finish intervals overlap, so they merge into $v_{4,5}$ by component-wise union, while χ is intersected: $\{j_3\} \cap \{j_2\} = \emptyset$. **5** j_4 becomes ready once both predecessors finish (its delays are zero), giving final bounds ($BCRT, WCRT$) of (1, 2), (3, 7), (2, 7), (4, 8) for j_1 – j_4 ; every LFT meets its deadline, certifying the job set.

of v , and the next dispatch has certainly happened once a core is certainly free with any of them certainly ready [2]. Because a non-preemptive job runs to completion, its *earliest* and *latest finish times* are $EFT_c = s_c^{min} + C_c^{min}$ and $LFT_c = s_c^{max} + C_c^{max}$. For each candidate, the analysis creates the successor state with updated D , A , and auxiliary information, and updates the response-time bounds via

$$BCRT_c \leftarrow \min\{BCRT_c, EFT_c - r_c^{min}\} \quad (1)$$

$$\text{and } WCRT_c \leftarrow \max\{WCRT_c, LFT_c - r_c^{min}\}. \quad (2)$$

At termination, these are the extrema of c 's response time over every dispatch of c anywhere in the graph.

Without merging, every dispatch order would produce its own path and the graph would grow with the number of dispatch orders. The merging phase combines similar-enough states into one. All four algorithms require merge candidates to satisfy (C1) equal dispatched sets D ; the remaining conditions compare the timing fields, requiring (C2) compatible availability intervals A_k and (C3) compatible auxiliary summaries where the variant has any. What compatible means is variant-specific: the uniprocessor analysis requires the states' single availability intervals to overlap; the DAG and gang analyses require A_k of one state to overlap A_k of the other, for every k ; and the EDD analysis requires each A_k pair and each finish

interval in F (matched per job) to overlap *or be contiguous* [5]. The merged state takes the component-wise union: each A_k becomes the smallest interval containing both inputs' A_k , likewise each interval in F , and, in the EDD analysis, χ becomes the intersection of the two sets.

The merged state represents every scenario of both inputs; this is why a state's incoming paths may dispatch the same set of jobs in different orders. Merging is sound but may also admit scenarios present in neither input (the widened intervals cover combinations of timing values that neither state reached). While it preserves exactness for the original analysis of sequential jobs scheduled by work-conserving JLFP policies on a uniprocessor [1], it is a source of pessimism in the other analyses, including the DAG, gang, and EDD variants [2–5].

The case for hardware acceleration. Merging curbs but does not tame the growth. Each layer can still spawn up to $|\mathcal{L}_i| \cdot |R|$ children, with R the candidate set of a state, so the number of states per layer can grow exponentially with the number of jobs. This growth is not an artifact of the SAG: schedulability analysis under release jitter and execution-time variability is coNP-hard already on one core, and at least as hard on multiple [13, 14]. Algorithmic optimizations such as partial-order reduction [11] can collapse redundant branches

and yield large speedups in practice, but they cannot alter the worst-case exponential scaling of the state space.

The resulting cost is visible in every analysis that exhaustively explores this state space. The most recent exact timed-automata encodings by Foughali et al. [9] for limited-preemptive periodic tasks on one core, extended by Hansen et al. [10] to parallel applications on multiple cores, verify an industrial case of seven tasks and 710 segments in under a minute, but their scalability experiments stop at 4 to 12 tasks on at most four cores, at 30% utilization, under timeouts of a few hundred seconds. The SAG makes the opposite trade in general: safe rather than exact bounds, in exchange for far larger workloads. Hansen et al. adopt it as the reference sufficient test [10]. Yet sequential implementations still take minutes to hours at moderate scale: one recent evaluation reports over 12,000 seconds for a SAG analysis of a 10-core workload at 60% utilization [12].

The bottleneck above is sequential execution, not the state space itself, and two properties of the SAG invite parallel exploration. First, the expansion of each (state, candidate) pair on a given layer is independent of every other pair on that layer, exposing data parallelism that scales directly with layer width. Second, merging operates entirely within a single layer, and condition C1 induces a natural partitioning: only states with equal dispatched sets D are mergeable, so the search for mergeable states parallelizes over the resulting partitions. These observations motivate the shift from purely algorithmic optimization to hardware-accelerated parallel execution.

III. FUNDAMENTAL BARRIERS TO A DIRECT GPU PORT

The two structural properties at the close of §II suggest that the SAG implementation can be directly ported to a GPU-aware implementation. Each (state, candidate) pair expands independently of every other within a layer, and once states are grouped by their dispatched set, the search for merge candidates parallelizes across the resulting groups. A canonical implementation would therefore assign one thread per state, enumerate the eligible candidates in parallel, and consolidate each layer through a hash-based merge before advancing.

However, this naive pipeline inevitably collapses. In this section, we expose four fundamental mismatches between how GPUs run code and how the SAG produces work. These obstacles are not mere implementation artifacts that can be resolved by incremental tuning; rather, they are inherent complexities that require rethinking the design from the ground up. We introduce the relevant GPU architectural concepts as needed, referring the reader to [15] for a fuller account.

Uneven work per thread. We first fix the vocabulary. The CPU acts as the *host*; it offloads work to the GPU, the *device*, by launching a *kernel*. A kernel is a single function of which the device runs one copy per *thread*, with the number of threads chosen at launch. The copies are identical; what distinguishes them is that each thread receives its own index, which the code uses to select the data that thread operates on. Threads are launched in *blocks* of up to 1024. The block is the

unit of placement; each block is assigned to a single *streaming multiprocessor* (SM), one of the GPU's many processors, and the threads of a block share their SM's on-chip memory.

Within a block, threads are not scheduled independently. The hardware groups them into *warps* of 32 threads with consecutive indices and issues instructions one warp at a time. The block is a software grouping whose size the programmer chooses; the warp is a hardware one, its width fixed by the hardware at 32. Under the Single-Instruction Multiple-Thread (SIMT) model, at each step all 32 threads of a warp execute the same instruction, each on its own data and each on its own execution unit of the SM. A warp is therefore less like 32 independent workers than like one worker with 32 hands, who must do the same thing with all of them.

Executing a warp in lockstep is what makes a GPU fast; one instruction fetch drives 32 execution units of arithmetic in parallel. It is also what makes a GPU fragile. When the threads of a warp reach a conditional and disagree on which branch to take, the hardware cannot run both branches at once. It runs one branch with the threads that chose it, while the rest sit idle, and then runs the other branch with the roles reversed. This is *branch divergence*. A warp whose 32 threads all take different paths executes them one at a time. In the worst case, such a warp may run up to 32× slower, and its parallelism is entirely lost. Performance on a GPU thus does not come from supplying many threads, but from supplying many threads that execute the same instruction at the same time.

The SAG hands the GPU exactly the workload the SIMT model handles worst. Within a single layer, states differ in which jobs they have dispatched and in the finish-time intervals they have reached; one state can therefore expose many ready successors while another exposes only a few. For parallel DAG [4] and self-suspending workloads [5], the gap widens further: two states sharing the same dispatched set can produce entirely different successors, because which jobs can dispatch next depends on when their predecessors finished, not just on which jobs are already dispatched.

The natural mapping assigns one thread to one state, so the variance in candidate counts turns directly into idle threads: a thread whose state has two candidates finishes and waits while its neighbor with twenty is still working. Each candidate then runs through a chain of conditional tests on predecessors, priorities, start time, and deadline, and each test gives the warp another chance to diverge. The speedup that the thread count promises on paper thus evaporates in practice.

Unpredictable memory growth. The on-chip memory of an SM holds kilobytes, but a layer under construction can run to gigabytes. A layer therefore lives in *global memory*, the device's own DRAM, commonly called VRAM. Global memory delivers its bandwidth only under two conditions.

The first condition concerns how data is laid out. The memory system serves a warp, not a thread. When the 32 threads of a warp read from 32 adjacent addresses, the hardware fetches the whole range at once (this is a *coalesced* access). When the same threads read from 32 scattered addresses, the same

volume of data can take up to 32 separate fetches. How fast a kernel reads memory is therefore governed not only by how much it reads but also by how the data is arranged, i.e., global memory is fast only when a warp can read it in stride.

The second condition concerns when memory is allocated. The host ordinarily sizes and allocates device buffers before the kernel launches, so the kernel receives pointers to storage that already exists. Allocating from inside a running kernel is possible (device code may call `malloc`), but it is slow, and it can fail, e.g., with thousands of threads drawing on one device heap, some thread eventually finds it empty and receives null. That thread has a computed result and nowhere to put it. It cannot wait, since nothing will free memory, and it cannot ask the host, which is unreachable until the kernel ends; all it can do is set an error flag and drop its result. The other threads allocated before the heap ran out, so their results are already written. When the kernel ends and the host reads the flag, it knows only that some results are missing, not which, so the launch must be discarded and rerun.

The SAG breaks both conditions. States have no uniform layout. In the DAG and EDD variants, a state keeps the finish-time interval of a dispatched job only while some successor of that job is still undispatched, and drops it afterward [4]; the set F of retained intervals therefore differs in size from state to state. Adjacent states in memory are thus variable-length records with fields at different offsets. A warp reading 32 such records cannot coalesce. Buffer sizes are also unknown before launch. How many successors a (state, candidate) pair produces depends, at run time, on precedence, priority, and interval overlap, so the size of layer $\mathcal{L}_{i+1}$ is unknown until layer $\mathcal{L}_i$ has finished expanding and merging.

Solving the memory problem is hard. Allocating as states are created is the slow and fragile path described above, whereas allocating for the worst case up front wastes most of the buffer on typical layers and outgrows VRAM.

The merge bottleneck. Branch divergence can be avoided by *warp-cooperative* algorithms, which invert how work is divided. Instead of giving each thread its own task and hoping the tasks are equally large, they give a single task to the whole warp and cut it into 32 equal pieces. The hardware makes the coordination cheap. Because the 32 threads of a warp execute together, a GPU lets them pass values directly to one another, without going through memory. In a single instruction, a warp can combine one value from each thread, say, to compute the minimum of 32 numbers or to learn which threads answered yes to a test. Rather than one thread per state, for example, all 32 threads take on one state together: the state’s candidates are dealt out one per thread, each thread checks its one candidate, and a single instruction then tells the warp which candidates passed.

SAG’s merge phase resists this pattern. A pairwise compatibility check between two states verifies three conditions from §II: equal dispatched sets (C1), pairwise overlap of all m core-availability intervals (C2), and pairwise overlap of every interval in F (C3). The three conditions touch three different

data structures of three different sizes: a bitmask of length $|J|$, a vector over m cores, and a set of $|F|$ intervals whose size itself varies from state to state. There is no even 32-way split of this work. However the warp distributes it, some threads run short, some run long, and the uniform per-thread workload that warp-cooperative algorithms depend on never materializes.

One might hope to sidestep the warp mismatch with a different parallel decomposition, but the difficulty runs deeper. Existing SAG implementations keep merging tractable by hashing states on their dispatched set D : a new state compares only against the few others in its bucket. But this approach is *inherently sequential*. Every time two states in a bucket merge, the merged state has wider intervals than either input, and whether a later arrival qualifies to merge depends on those widened intervals. Whether state k ends up merged, and with which others, therefore depends on which of the preceding $k-1$ states already merged together, and on how their intervals grew along the way. No parallel ordering of the merges can respect this dependence without serializing the phase. Putting merging on the GPU means reformulating the problem, not porting the sequential algorithm.

The host-device gap. Host and device keep separate physical memories, joined by a PCIe link. The link carries tens of gigabytes per second, whereas the GPU reads its own DRAM at terabytes per second, orders of magnitude faster. Whatever crosses the link is therefore expensive relative to whatever stays on the device. Kernel launches impose a second, smaller fixed cost. Each takes a few microseconds of overhead before the first thread runs. For a kernel that occupies the whole device for a millisecond, that overhead is invisible, but for one that finishes in ten microseconds, it dominates.

The SAG runs into both costs. It advances layer by layer, and shipping each layer back and forth across the link saturates the link long before the GPU’s compute is busy. And many layers, especially at the start and end of the construction, contain too few states to amortize the cost of launching a kernel; on those layers, GPU execution is strictly slower than running everything on the CPU.

IV. THE SAGA FRAMEWORK ARCHITECTURE

The previous section showed that a direct port of the SAG to the GPU fails on four counts. SAGA is designed around all four at once. This section describes its orchestration layer, which resolves two of the obstacles at the architectural level: unpredictable memory growth and the host-device gap. The remaining two are resolved inside the operators the orchestration invokes: the warp-cooperative expansion kernels of §V (uneven work per thread) and the sort-based merge pipeline of §VI (the merge bottleneck). These operators are pluggable: they touch the architecture only through a small set of GPU-resident interfaces, so a different SAG variant is obtained by substituting them, with the orchestration layer untouched.

SAGA processes the SAG layer by layer, as the analysis requires (§II), and decomposes each layer into *waves* of bounded size. A wave is the unit of GPU work: one kernel

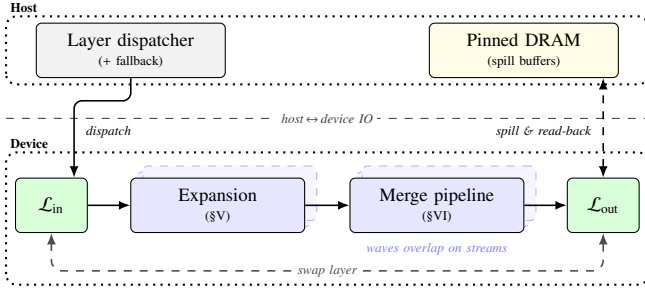


Fig. 2: SAGA framework architecture.

launch for eligibility evaluation, one for successor creation, and a short pipeline of kernel launches for the per-wave merge. A layer completes when every wave has been processed and its merged states written back to the layer-resident buffer. Three orchestration mechanisms span this structure: wave streaming with GPU persistence, layer residency with host-side overflow, and adaptive CPU fallback for narrow layers. Fig. 2 shows the three working in concert around the wave pipeline of a single layer.

Wave-based streaming with GPU persistence. This mechanism removes the interconnect half of the host-device gap (§III): a layer-at-a-time port would ship each layer back and forth across the link, twice per layer, saturating it long before the GPU’s compute is busy. SAGA instead keeps the entire layer resident in a pair of GPU-persistent buffers that alternate as input and output across consecutive layers. Within a layer, waves read a contiguous slice of the input buffer, produce expanded states into per-wave scratch regions, and deposit the merged result into the buffer of the next layer. During ordinary layer processing, only small per-layer meta-data crosses the interconnect (such as wave merge counts and the final response-time bounds of the analyzed jobs).

Finally, two CUDA streams run consecutive waves in a double-buffered schedule, i.e., while one computes a wave, the other prefetches the next wave’s input and drains the previous wave’s output. Hence, inter-wave data movement overlaps with compute and the kernels stay continuously fed.

A wave could seemingly overrun its scratch, but its memory demand has a static bound. A wave spans at most w states; a state emits at most one record per undispatched job, hence at most $|J|$; and each record yields one successor. Let B_{succ} be the bytes a wave sets aside per successor, i.e., the record, the successor state, and the sort key and index the merge consumes. The worst-case demand of a wave is then

$$B_{\text{wave}}(w) = w \cdot |J| \cdot B_{\text{succ}} + B_{\text{sort}}, \quad (3)$$

with B_{sort} the scratch reserved for the sort routine.

The workload model fixes $B_{\text{succ}} = B_{\text{rec}} + B_{\text{state}} + B_{\text{key}}$, one term per item of that list. In the EDD instantiation, a record holds a state index, a job index, and the candidate’s start-time bounds. A state holds three $|J|$ -bit masks (D , the certainly-executing set χ , and the jobs present in F), the m availability intervals, a finish-interval table of capacity F_{max} , and two

counters. The merge’s sort key is D itself; B_{key} covers it and the accompanying index and boundary entries, double-buffered by the merge. Each term is the sum of these fields at the widths the implementation fixes, so B_{succ} is determined by $|J|$, m , and F_{max} alone. The other three instantiations change only the contents of a record and of a state. F_{max} is a configured capacity, and a workload that would exceed it is reported rather than analyzed. Fixing F_{max} also gives every state the same length: where §III had states as variable-length records with fields at different offsets, state i now begins at offset $i \cdot B_{\text{state}}$, so a warp reading consecutive states coalesces. The padding this costs is charged in Eq. (3).

The host evaluates B_{succ} once the input files have been parsed and the precedence tables built, before any device buffer is allocated and before the first layer is processed. The wave width w is then fixed against a configured scratch budget; the wave’s buffers are allocated at that width once and never resized, and Eq. (3) is what certifies that no wave can overrun them. Reserving this way is out of reach a layer at a time, where the worst-case width is dictated by the state space (§III).

Layer residency and host-side overflow. Wave sizing keeps the resident footprint small; providing capacity for a layer that outgrows the resident buffers is this mechanism’s job. SAGA extends residency off-device through *pinned* host memory, i.e., memory whose pages are locked in the host’s physical RAM, which the GPU can read and write directly across the PCIe link, so copies to and from it overlap with kernel execution instead of stalling it. Each of the two layer buffers is paired with a pinned DRAM spill buffer, and each spill buffer alternates in step with the layer buffer it is paired with: the pair that receives $\mathcal{L}_{i+1}$ as it is produced is the pair the wave pipeline reads one layer later, when $\mathcal{L}_{i+1}$ becomes the input. Producer and consumer are the two sides of that same pipeline. Its output side writes $\mathcal{L}_{i+1}$ and spills whatever does not fit in the resident buffer; its input side reads $\mathcal{L}_i$, which overflowed the same way one layer earlier, so staging a wave’s input pulls the spilled part of $\mathcal{L}_i$ back from that layer’s own spill buffer. The two sides are always a layer apart, hence on different pairs, and never contend for one. In effect, a layer is a logically contiguous staging area whose physical storage spans VRAM and DRAM, with the split determined at run time by the producing layer’s size. A workload whose layers fit in device memory incurs no overflow traffic; a workload that overflows pays only for the states that do not fit.

Adaptive CPU fallback for narrow layers. The launch-overhead half of the host-device gap (§III) remains: the analysis opens and closes with layers of a handful of states, where kernel-launch overhead dominates the work and warp-level parallelism sits unused, so a modest CPU implementation beats the GPU outright. SAGA therefore monitors layer size and dispatches any layer below a configurable threshold to the CPU: the layer is copied to host memory, expanded and merged there by the original sequential SAG implementation [7]. The result is copied back to VRAM before the next GPU-dispatched layer begins. Wide layers amortize the launch

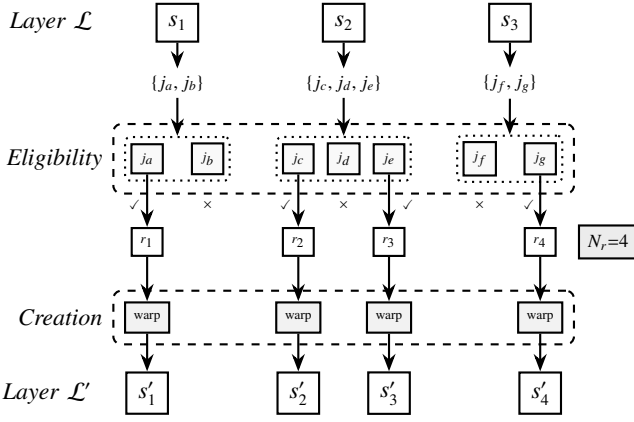


Fig. 3: The expansion pipeline. Each state s in $\mathcal{L}$ has a candidate list of jobs that could be dispatched next, e.g., $\{j_a, j_b\}$ for s_1 . The eligibility kernel runs one warp per state; inside each warp (dotted box), one thread examines one candidate. Each thread whose candidate is eligible (✓) emits a record r_k . The creation kernel reads the record count N_r , runs one warp per record, and writes a successor to $\mathcal{L}'$.

cost and profit from massive parallelism; narrow layers are strictly better served on the CPU. Each layer thus runs on the processor matched to its size, and neither stalls on the other.

V. PARALLEL EXPANSION: ELIGIBILITY AND CREATION

The expansion step transforms a layer $\mathcal{L}$ of states into a multiset $\mathcal{L}'$ of successors (§II). Two of its properties, already noted there, admit a parallel decomposition. First, the candidate set of a state $s \in \mathcal{L}$ is determined entirely by s and the workload parameters, independent of every other state in $\mathcal{L}$. Second, for a candidate j of s (we call (s, j) a *feasible pair*), the successor s' is a function of s , j , and the workload parameters alone. Expansion therefore splits into a pipeline: an *eligibility* stage maps each state to its feasible pairs, and a *creation* stage maps each feasible pair to its successor.

Each stage is realized as a GPU-resident kernel. Fig. 3 shows the two-kernel pipeline. The two kernels communicate through a single device-side counter, written by the eligibility kernel and read by the creation kernel, so no host-device round trip separates them. The remainder of this section gives the work mapping of the two stages and the parallel primitives they require, the structural pre-filter that keeps provably non-eligible candidates out of the sweep, and the equivalence of the parallel expansion with the sequential one.

Work mapping. §III showed that the natural thread-per-state assignment collapses under the variance in candidate-set size across a layer. At the layer granularity, different states admit different numbers of candidates; at the per-candidate granularity, different candidates incur different numbers of predicate evaluations. A mapping that binds a single thread to either unit of work exposes the full variance as divergence. We avoid this by assigning one *thread group* per unit of work; in our NVIDIA-based implementation, this group corresponds

to a warp (§III). In the eligibility stage, a single warp processes one state: its threads cooperatively iterate over the candidate list, evaluate one candidate per thread in lockstep, and combine per-thread contributions via warp reductions. In the creation stage, a single warp processes one feasible pair: the threads cooperatively perform the multi-component successor construction (updating the per-core availability intervals, pruning the finish-time set F , and inserting the new finish-time record).

The mapping relies on three warp-level primitives. A *ballot* gathers one bit from each thread into a mask all of them receive. A *reduction* combines one value per thread under an associative operator. A *broadcast* hands one thread's value to the rest. Coordination beyond a single warp additionally uses global *atomics*: a read-modify-write that the GPU guarantees to be indivisible, even across warps.

Eligibility. The eligibility kernel reads the input layer and produces a dense stream of eligibility records, each carrying a state index, a job index, and the start-time bounds $[s_j^{\min}, s_j^{\max}]$ of the pair. The kernel proceeds in two sub-phases.

First, the warp constructs the set R_{struct} of jobs passing the structural test of §II at s , and computes for each $j \in R_{\text{struct}}$ the latest moment at which the scheduler could observe j becoming ready under any scenario consistent with s (its readiness bound $\rho_j^{\max}$). Warp reductions over the $\rho_j^{\max}$ then supply the two minima that the moments of §II are built from: the minimum over R_{struct} , the earliest at which some job is certainly ready, and, for each j , the minimum over the jobs of higher priority than j .

Second, the warp revisits each $j \in R_{\text{struct}}$ and evaluates $[s_j^{\min}, s_j^{\max}]$ under the timing test, which bounds when a candidate could start as the next dispatch. A ballot collects the outcomes of the tests the warp ran in lockstep. If the interval is non-empty, an eligibility record is appended to the output stream by an atomic increment of the shared record counter N_r .

Pre-filtering. Several jobs can be rejected before the second sub-phase above. The SAG literature provides sound conditions for such early rejection, all of the same shape: a job cannot be dispatched next unless certain other jobs have been dispatched already, e.g., its predecessors in the DAG and EDD variants [4, 5], or the jobs that dominate it in priority order [11]. Our contribution is not a new condition but a uniform way to install any of them in the eligibility kernel.

Every condition of this shape reduces to one test. Precompute, for each job j , the *prerequisite* set $\text{Req}(j)$ of jobs that must precede it, stored as a $|J|$ -bit mask over the job set; at run time, j survives only if $\text{Req}(j) \subseteq D$, a word-wise test costing $\lceil |J|/64 \rceil$ bitwise operations against the parent state's dispatched set. Several conditions can be installed at once, as separate tables: a job must pass all of them, in any order. The kernel itself never changes — it sees only bits, and the correctness of each condition rests with the paper that introduced it, so future rejection conditions from the SAG literature can be added by precomputing one more table.

The test also fits the sweep it extends: each thread applies it to its own job in sub-phase one, so the interval arithmetic

of sub-phase two runs only for the jobs that pass, and the warp’s reads of the Req tables coalesce, since each job’s mask occupies $\lceil J/64 \rceil$ consecutive words (§III). The survivors are also more homogeneous than the nominal candidate list, so the chain of conditional tests in sub-phase two gives the warp fewer opportunities to diverge.

Creation. The creation kernel reads the eligibility stream. The number of warps it launches comes directly from the device-side counter N_r that the eligibility kernel updated, so the stage is sized to the actual number of feasible pairs, not a worst-case bound, and no host-device synchronization separates the two stages within a wave.

For each eligibility record (s, j) , the assigned warp constructs the successor s' cooperatively. The dispatched set D' adds j to a copy of D . The finish-time set F' is extended with j ’s finish bounds and pruned: an entry is retired when its finish time can no longer constrain a later dispatch, a test evaluated by a reduction across the warp. The availability vector A' is obtained by inserting the pair’s finish interval into the sorted core intervals and removing dominated entries in the same pass.

Creation performs two further updates. First, the per-job bounds $BCRT[j]$ and $WCRT[j]$ are updated by global atomic minimum and maximum against the pair’s contribution. The bounds at the end of the analysis are therefore the extrema of j ’s response time over every dispatch of j anywhere in the graph. Second, the successor itself is appended to the wave’s output buffer by an atomic increment of a shared counter, which reserves a contiguous range of the buffer for it. A broadcast hands the start of that range to the warp’s threads, which then divide the fields of s' among themselves and write them into the range in parallel.

Equivalence. The two-stage parallel expansion produces the same multiset of successor states, and the same response-time bounds, as the sequential expansion. Three properties ensure this. First, no races: a successor is a function of its record (s, j) and the static workload parameters alone (§V), each feasible pair is assigned its own warp, and no mutable structure is shared between warps during creation. Second, no schedule sensitivity: within a warp, values combined across threads use only commutative and associative operators (minimum, maximum, bitwise OR), and every other step is a deterministic computation on read-only inputs under a fixed thread-to-task assignment, so the successor’s fields are the same under any thread schedule; across warps, the only shared writes are the atomic minima and maxima into $BCRT$ and $WCRT$, whose results are interleaving-independent for the same reason. Third, no lost or duplicated work: the eligibility kernel evaluates every (s, j) pair not rejected by one of the pre-filters installed above, and each atomic increment returns a distinct range, so distinct feasible pairs occupy distinct positions in the stream the creation kernel then processes in full. The one quantity that is not preserved is the *order* of the emitted records, which varies run to run with the hardware schedule; the multiset is invariant, the sequence is not. §VI begins exactly there.

VI. PARALLEL MERGE: SORT-BASED DECOMPOSITION

The merge phase consolidates the states produced by the parallel expansion of §V into a smaller layer by collapsing compatible states into a single abstract representative. The compatibility conditions C1–C3, defined per variant in §II, require equal dispatched sets and compatible interval fields; we write $s_1 \sim s_2$ when all hold. Two compatible states may be replaced by the component-wise union of their interval fields (§II), and the resulting *merged* state is sound for every execution scenario abstracted by either of the two states.

This phase is the hardest obstacle to GPU parallelization, as the merge is a greedy procedure whose outcome depends on the order in which states are processed, and, as §V closed by noting, the parallel expansion delivers the layer in an order that varies from run to run. This order-dependence is not introduced by parallelization; it is a property of the merge itself. Different orderings yield different but sound results; the sequential implementation induces one ordering, SAGA induces another. What no data-parallel schedule can do is reproduce the sequential ordering’s outcome without serializing; what it can do is fix an ordering of its own and remain sound. We therefore replace the sequential merge with a *sort-based decomposition* whose every stage is data-parallel.

The decomposition has three stages. A *sorting* stage arranges the states contiguously by their dispatched set, so that states sharing a D (and thus satisfying C1) occupy adjacent positions. A *grouping* stage detects the boundaries between distinct D values and partitions the layer into per- D groups. A *slot-search* stage partitions each group into slots under the residual conditions C2 and C3, and collapses each slot to a single merged state. Fig. 4 summarizes the pipeline.

Sorting and group boundary detection. Sorting a layer of states by their dispatched set is a standard parallel task. Each state is represented by a (D, index) key-value pair. The sort preserves the original indices, so the full state records are accessed indirectly through the sorted index array. Our implementation uses GPU-resident radix and merge sort routines, but any GPU-resident parallel sort suffices for this stage.

Once states are sorted by D , group boundaries are detected by a parallel comparison kernel. Thread i emits a boundary flag if the D of the i -th sorted state differs from that of the $(i-1)$ -th. A parallel exclusive *scan* then converts the boundary flag array into a per-state group identifier, and a *compaction* step collects the group start positions and sizes. These are all standard device-parallel primitives. After this stage, the layer is represented as a list of groups, each containing all states that share a single D and is ready for the slot search.

Parallel slot search. Within a group, all states satisfy C1 by construction. The remaining task is to partition the group into slots under C2 and C3 and collapse each slot to a representative. The sequential merge processes the group one state at a time and queries an incrementally maintained slot list. In practice, the slot list of a group is typically short, holding substantially fewer slots than the width of a warp. This admits a fully parallel alternative: the compatibility of

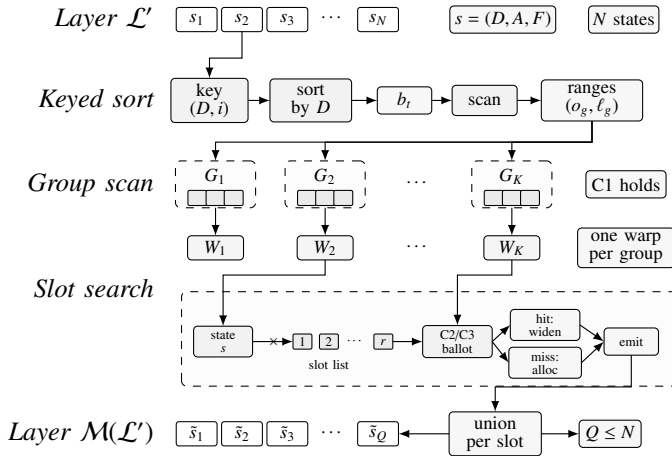


Fig. 4: The sort-based merge pipeline. The post-expansion layer $\mathcal{L}'$ is converted to (D, index) key-value pairs, sorted by the dispatched set D , and scanned using boundary flags (b_i) to recover contiguous per- D groups, represented by their start offsets and lengths (o_g, ℓ_g) (sorting enforces C1). Each group is assigned to a warp, which maintains the slot list in shared memory; a candidate state is tested against all active slots in parallel under C2 and C3, then either widens a compatible slot or allocates a new one. The emitted slot representatives form the merged layer $\mathcal{M}(\mathcal{L}')$.

a candidate state against *all* slots currently in the list can be tested concurrently, with one thread assigned to each slot.

We realize this by assigning one warp to each per- D group. The warp holds its group's slot list in *shared memory*, the on-chip memory of the SM introduced in §III, which the programmer manages explicitly. For each incoming candidate state, each thread evaluates C2 and C3 against the slot it was assigned. A ballot across the warp then identifies which slots are compatible. The threads cooperatively widen the interval bounds of the lowest-indexed compatible slot with those of the candidate. If no slot is compatible, a new one is allocated at the end of the list and the candidate copied in. When a slot list outgrows the warp, the search proceeds in rounds, the candidate broadcast to each round and the parallel test repeated.

Soundness. Unlike expansion, where §V established equivalence with the sequential algorithm, the merge phase is order-dependent even in its original sequential formulation. The appropriate guarantee is *soundness*: the merged layer abstracts every scenario represented by the input layer, whichever slot partition the ordering induces. We make this precise.

Definition 1 (Dispatch prefix). *Given a scenario γ , its dispatch prefix is the ordered sequence of jobs dispatched along γ 's path through the graph; the prefix matches a dispatched set D if it contains exactly the jobs of D , in any order.*

The order-indifference is deliberate: a merged state's incoming paths may dispatch D in several orders (§II), and a scenario reaching the state along any of them must count as represented.

Definition 2 (Scenario abstraction). *A state s represents an execution scenario γ if the interval-valued parameters of γ lie within the interval bounds of s and the dispatch prefix of γ matches the dispatched set D of s . A layer $\mathcal{L}^*$ soundly abstracts a layer $\mathcal{L}$ if every scenario represented by any state in $\mathcal{L}$ is represented by some state in $\mathcal{L}^*$.*

Lemma 1 (Soundness of merge operator). *Let s_1, s_2 be two states satisfying C1, C2, and C3, and let $s^* = \text{merge}(s_1, s_2)$ denote their component-wise union. Every scenario represented by s_1 and every scenario represented by s_2 is represented by s^* .*

Proof. Condition C1 gives $D_1 = D_2$, so s^* inherits the common dispatched set $D^* = D_1 = D_2$, and a dispatch prefix matching D_1 or D_2 matches D^* (Definition 1), preserving the prefix condition of Definition 2. Each interval-valued field of s^* is, by construction, the union of the corresponding fields of s_1 and s_2 , so every interval bound of s^* contains the corresponding bound of either input. The set of scenarios represented by a state is monotone in its interval bounds: widening can only include additional scenarios. Every scenario represented by s_1 or s_2 is therefore represented by s^* . $\square$

Lemma 1 is inherited from the sequential SAG literature. We restate it as the soundness of our decomposition rests on it.

Theorem 1 (Soundness of the parallel merge). *Let $\mathcal{L}'$ be a post-expansion layer, and let $\mathcal{M}(\mathcal{L}')$ be the layer produced by the sort-based decomposition of this section. Then $\mathcal{M}(\mathcal{L}')$ soundly abstracts $\mathcal{L}'$.*

Proof. The decomposition partitions $\mathcal{L}'$ into per- D groups via sorting and boundary detection, which reorder states but do not alter them, and then into slots $S_1, \dots, S_k$ via the parallel slot search; $\mathcal{M}(\mathcal{L}')$ consists of one representative $\tilde{s}_i$ per slot, the iterated component-wise union of the states in S_i . Fix $s \in \mathcal{L}'$ and let S_i be the slot containing s . Every pairwise merge within S_i combines a state with the current slot representative under C1, C2, and C3 (the slot search admits a state only after testing C1–C3 against the representative's current, possibly widened, bounds), so Lemma 1 applies at each step, and by induction every scenario represented by any state in S_i , including s , is represented by $\tilde{s}_i \in \mathcal{M}(\mathcal{L}')$. This holds for every $s \in \mathcal{L}'$; the slot-search ordering affects only the partition $\{S_1, \dots, S_k\}$, not whether Lemma 1 applies at each pairwise merge. $\square$

Remark 1 (Non-transitivity of the compatibility relation). *The compatibility relation $\sim$ defined by C1, C2, C3 is reflexive and symmetric but not transitive (Fig. 5), the same structure that makes state minimization of incompletely specified finite-state machines NP-complete [16].*

Non-transitivity leaves the merged layer an artifact of the order in which pairs are attempted. It is natural to ask whether some ordering is *distinguished*, i.e., an optimal one that every implementation, sequential or parallel, would be obliged to obey. This question matters: if such an ordering existed and were affordable, and the sequential SAG used it, a parallel

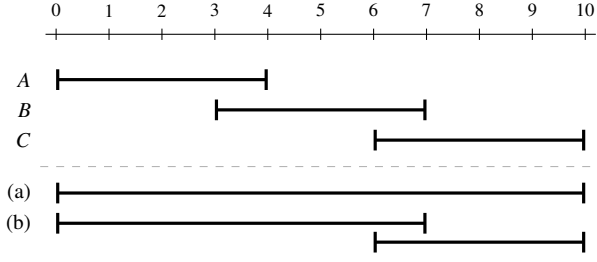


Fig. 5: Order-dependence of merging under non-transitive compatibility. Consider three states with availability intervals $A = [0, 4]$, $B = [3, 7]$, and $C = [6, 10]$ in the same per- D group on a single core (making condition C3 trivially satisfied). (a) Processing in the sequence A, B, C yields a single merged slot: state B merges into the slot opened by A , widening its interval to $[0, 7]$, which subsequently overlaps with and absorbs C to $[0, 10]$. (b) Processing in the sequence A, C, B yields two distinct slots: state C lacks overlap with the initial interval $[0, 4]$ of A and opens a new slot $[6, 10]$; state B overlaps with both existing slots but merges into the first compatible slot it evaluates, widening the slot opened by A to $[0, 7]$.

merge would either match it or be strictly more pessimistic. Theorem 2 rules this out for any polynomial-time analysis.

Theorem 2 (Hardness of merge ordering). *Minimizing the number of states of a SAG over its merge orderings is NP-hard, even when merging does not widen interval bounds.*

Proof. We first establish hardness for a single layer, then lift it to the entire graph. We characterize the minimum graph-theoretically, then reduce from minimum clique cover. Let $G_{\sim}(\mathcal{L}')$, the *compatibility graph* of $\mathcal{L}'$, be the simple undirected graph with vertex set $\mathcal{L}'$ and edges $\{\{s_i, s_j\} : s_i \sim s_j\}$, and let $\mathcal{M}_{\pi}(\mathcal{L}')$ denote the layer produced by a merge ordering π run until no compatible pair remains. We work under the relaxation in which the merge operation does not widen interval bounds, so that admission of a state into a slot requires pairwise compatibility with every existing member of that slot, rather than compatibility with a widened representative. Under this relaxation, every slot of $\mathcal{M}_{\pi}(\mathcal{L}')$ is a pairwise-compatible subset of $\mathcal{L}'$, hence a clique of $G_{\sim}(\mathcal{L}')$; conversely, any clique cover of $G_{\sim}(\mathcal{L}')$ is realized by an ordering that processes one clique at a time. Therefore $\min_{\pi} |\mathcal{M}_{\pi}(\mathcal{L}')| = \theta(G_{\sim}(\mathcal{L}'))$, the clique cover number, equivalently $\chi(G_{\sim}(\mathcal{L}'))$, the chromatic number of the complement.

We reduce minimum clique cover, which is NP-hard on arbitrary simple graphs [17], to the optimal-ordering problem. Given an arbitrary graph $H = (V_H, E_H)$, we construct a layer $\mathcal{L}_H$ such that $G_{\sim}(\mathcal{L}_H) \cong H$. Encode each vertex $v \in V_H$ as a state s_v , and assign all states the same dispatched set so that C1 holds among every pair. For each non-edge $\{u, v\} \notin E_H$, reserve a fresh dimension d_{uv} of the compatibility test (a core of C2 or an F entry of C3) in which s_u and s_v are assigned disjoint intervals, while every other state pair is

assigned overlapping intervals in d_{uv} . Remaining dimensions are assigned universally compatible bounds. The total number of dimensions required is $O(|V_H|^2)$, which the SAG framework accommodates when the workload exposes sufficient cores or F entries. By construction, $s_u \sim s_v$ in $\mathcal{L}_H$ iff $\{u, v\} \in E_H$, so $G_{\sim}(\mathcal{L}_H) \cong H$ and $\min_{\pi} |\mathcal{M}_{\pi}(\mathcal{L}_H)| = \theta(H)$. An oracle for the optimal-ordering problem therefore solves minimum clique cover, and NP-hardness carries over.

Now place $\mathcal{L}_H$ in the terminal layer, which has no successors, and let the earlier layers be free of compatible pairs, so that they contribute a constant Σ . The graph size is then $\Sigma + |\mathcal{M}_{\pi}(\mathcal{L}_H)|$, and the hardness extends to the entire graph. $\square$

Theorem 2 places optimal merge ordering in the broader NP-hard family of axis-parallel box partitioning problems [18]. Every SAG implementation is therefore a polynomial-time greedy algorithm under a fixed ordering, not an optimization procedure. Because no optimal ordering is available at polynomial cost for a parallel implementation to obey, how exhaustively to merge becomes a system design choice balancing analytical precision against parallel workload: aggressive collapsing widens representatives and propagates pessimism (Lemma 1), whereas retaining more distinct slots incurs parallel work that the GPU readily absorbs through warp-cooperative testing. The deviations reported in §VII are accordingly the difference between two sound abstractions of the same already-pessimistic analysis, not a loss of exactness. Empirically, the observed relative difference in per-job worst-case response times between the parallel and sequential merges is bounded by 0.2%.

VII. EVALUATION

We evaluate SAGA against the EDD analyzer of Srinivasan et al. [5], as implemented in `nptest` on SAG-org [7]. We pin the baseline at v3.1.7, the release current when the measurements were taken, so that the comparison remains reproducible as the repository advances. The EDD variant is the most computationally demanding member of the SAG lineage. Under shared workload parameters, its median analysis wall time exceeds that of the other variants we instantiate [1, 3, 4] by at least two orders of magnitude. The evaluation spans three axes: *speedup*, *accuracy*, and *cross-platform portability*. To assess these metrics, synthetic task sets are generated following the UUniFast-derived methodology of the original SAG papers.

Experimental setup. Table I summarizes the experimental setup, including the hardware platforms, baseline tool, and workload generation parameters. The framework runs the same binary across all three GPUs with no architecture-specific code paths; only the GPU device changes between runs.

Wall-time and speedup on B200. In Fig. 6, the `nptest` median follows an exponential trend, exceeding 50 s at $n = 16$ and saturating the cap at $n = 32$. The B200 median exhibits a piecewise behavior: a fixed-cost regime, driven by CUDA-context, kernel-launch, and per-layer sort overheads, persists through $n = 28$, followed by an exponential growth regime

TABLE I: The hardware platforms used throughout, and the CPU baseline and workload generator of the EDD experiments. Table V gives the corresponding settings for the other variants.

Hardware and software setup	
Primary GPU	NVIDIA B200 (192 GB HBM3e)
Comparison GPUs	NVIDIA H100 (80 GB HBM3); NVIDIA A100 (40 GB HBM2)
CUDA toolkit	CUDA 12.8
Host	Intel Xeon Platinum 8592, 4 TB DDR5
CPU baseline	EDD analyzer [5] in npctest v3.1.7 [7]
Workload generator: UUniFast	
Tasks per set (n)	{16, 20, 24, 28, 32, 36, 40, 44, 48}
Cores (m)	$n/4$ (4:1 ratio)
Total utilization (U')	{20, 40, 60, 80}%
BCET / WCET ratio (β)	0.5
Segments per task (S)	5
Deadline type	implicit ($D = T$)
Priority assignment	deadline-monotonic
Replicates	25 fresh seeds per (n, m, U') point (100 task sets per n)

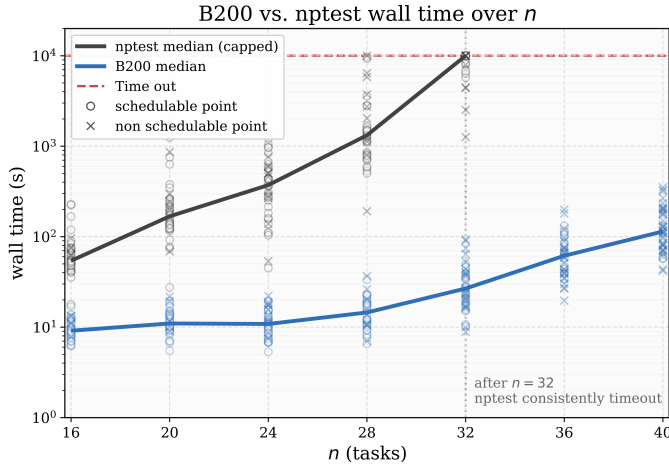


Fig. 6: Wall times of the framework on B200 and of npctest, over $n \in [16, 40]$. Markers summarize 100 task sets per n , with circles indicating schedulable task sets and crosses non-schedulable; solid lines show per- n medians and the dashed line is the 10,000 s harness cap.

reaching ~ 28 s at $n = 32$ and ~ 105 s at $n = 40$. The ratio of these two trends is the cap-bounded median speedup (Fig. 7), which increases through $n \in [16, 32]$ to $\sim 15\times$ at $n = 20$, $\sim 34\times$ at $n = 24$, $\sim 91\times$ at $n = 28$, and $\sim 470\times$ at $n = 32$. Past $n = 32$, npctest consistently times out within the 10,000 s harness cap on every task set in the grid; direct wall-time comparison is no longer meaningful. The solid curve declines past this point only because its numerator is fixed at the cap while the B200 denominator continues to grow. The dashed extrapolation extends npctest’s pre-cap exponential trend beyond the harness ceiling and divides by the measured B200 wall, exceeding three orders of magnitude near $n = 40$ and reaching approximately $2,100\times$ at $n = 48$.

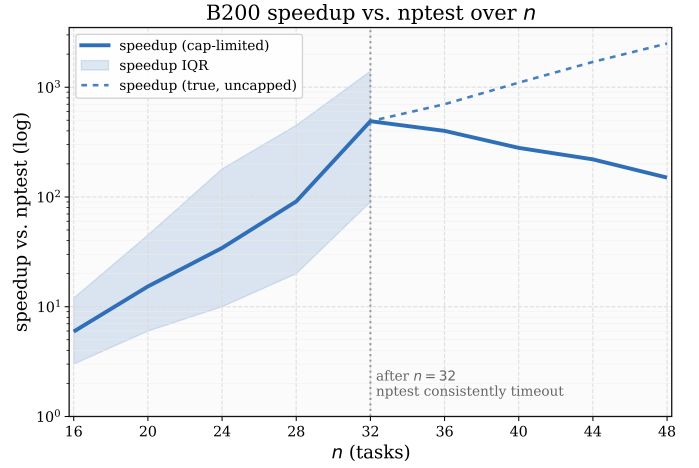


Fig. 7: B200 speedup over npctest, log scale. Solid: cap-bounded measured ratio. Shaded band: inter-quartile range. Dashed: extrapolated ratio with the cap removed.

TABLE II: Per-job comparison with npctest. “Both SCHED” is the subset of the 100 task sets at each n in which both tools converge to SCHED within the npctest harness cap. “Avg WCRT/BCRT” is the mean absolute per-job relative deviation $|X_j^{\text{GPU}} - X_j^{\text{npctest}}|/X_j^{\text{npctest}}$, averaged over every such job.

n	Sets	Both SCHED	avg WCRT (%)	avg BCRT (%)
16	100	82	0.05	0.02
20	100	71	0.09	0.04
24	100	46	0.14	0.07
28	100	29	0.17	0.09
32	100	14	0.19	0.11

Accuracy against the baseline. Across all 500 task sets at $n \in \{16, 20, 24, 28, 32\}$, SAGA and npctest return identical schedulability verdicts. We compare per-job WCRT and BCRT bounds against npctest across all configurations within the baseline’s tractability limit, namely $n \in \{16, 20, 24, 28, 32\}$.

As detailed in Table II, across the 242 task sets in which both tools converge to SCHED (25,480 jobs total), the mean per-job WCRT and BCRT deviations grow mildly with n , from 0.05% to 0.19% for WCRT and from 0.02% to 0.11% for BCRT. The deviation reflects the parallel and sequential algorithms traversing different greedy merge orders under a non-transitive compatibility relation; the speedup figures below therefore reflect a change of execution, not of abstraction.

Cross-GPU generalization. At $n = 16$ the three medians (Table III) are $5.02\times$ on A100, $5.30\times$ on H100, and $5.93\times$ on B200. At this small workload scale, execution time is strictly bounded by the CPU fallback for narrow layers and the fixed CUDA-context and kernel-launch costs for GPU-dispatched layers, which together mask the underlying architectural differences across platforms. The B200/A100 ratio then grows from $1.13\times$ at $n = 20$ to $2.35\times$ at $n = 24$, as older platforms enter their exponential regime before B200. At $n = 32$ the medians reach their maxima of $127\times$ on A100, $244\times$ on H100,

TABLE III: Median per-task-set speedup ($\times$) over `nptest`.

GPU	$n = 16$	$n = 20$	$n = 24$	$n = 28$	$n = 32$
B200	5.93	15.3	34.3	91.0	470.6
H100	5.30	14.9	17.8	60.6	244.7
A100	5.02	13.5	14.6	41.9	127.1

TABLE IV: Resource utilization (approximate %). VRAM: peak per-layer footprint relative to the per-buffer cap; Spill: fraction of waves spilling to pinned host memory.

n	A100 (40 GB)		H100 (80 GB)		B200 (192 GB)	
	VRAM	Spill	VRAM	Spill	VRAM	Spill
16	7	14	4	12	1	10
20	14	19	7	14	3	11
24	26	32	13	19	5	13
28	51	64	24	31	11	17
32	98	96	49	62	20	25

and 470 $\times$ on B200.

Resource utilization. In Table IV, the per-layer footprint (VRAM columns) depends on the workload and not on the hardware; the separation between platforms reflects cap differences alone. A100 (40 GB) reaches its cap at $n = 32$, H100 (80 GB) reaches $\sim 50\%$ of its cap at the same n , and B200 (192 GB) remains below 25% throughout. The spill trigger rate (Spill columns) follows the footprint-to-cap pressure: A100 crosses 50% at $n = 28$ and approaches 100% at $n = 32$, H100 reaches $\sim 60\%$ at $n = 32$, and B200 remains below 30% throughout. Consequently, the performance bottleneck of the SAG analysis fundamentally shifts: it is no longer bound by compute time, but rather constrained by VRAM capacity and host-device memory bandwidth.

Cross-variant validation. To substantiate generality beyond the EDD variant, we run the earlier SAG algorithms we instantiate on byte-identical datasets through both SAGA and the v3.1.7 SAG-org baseline [7] of each variant: ten seeds per variant, at the most demanding parameter setting where the baseline still completes within the 10,000 s cap (Table V). All 30 schedulability verdicts agree with the baseline. On an A100 — the weakest of our three GPUs, so these figures are conservative — speedups range from $\sim 62\times$ (median, uniprocessor) to $\sim 156\times$ (mean, moldable gang). Each GPU time is the median of ten idle runs. Table V also reports the current RTSS’24 EDD result for context.

VIII. RELATED WORK

The only direct precedent for SAGA dates back to 2013, when Ahmed et al. [19] parallelized EDF feasibility checking on a GPU. SAGA advances that decade-old baseline on both foundation and scope: it accelerates the SAG state-space abstraction rather than direct EDF enumeration, and it spans four published SAG algorithms rather than a single feasibility test. Parallel hardware reached scheduling-related computation earlier still, when Goossens and Baruah [20] used a multiprocessor for the preprocessing behind on-line

TABLE V: Unified SAG-lineage workload settings and modern-baseline results over co-completing datasets. The periodic RTSS’24 workload uses $U' = 40\%$ and $\beta = 0.5$.

	RTSS’17 uniproc. NP	ECRTS’19 LP-DAG	ECRTS’22 moldable gang	RTSS’24 self-susp. EDD
Tasks per set (n)	48	14	28	32
Cores (m)	1	4	4	8
<i>Results (modern CPU baseline vs. A100)</i>				
CPU time (s) mean	2,928	3,719	3,345	1,584
GPU time (s) mean	46.8	54.4	29.3	6.4
Speedup ($\times$) mean/median	72.8/62.2	68.3/68.4	156.2/95.4	456.0/127.1

admission control; what is parallelized there is an offline step feeding an acceptance test, not the exhaustive exploration and merging of a schedulability state space, which is why we regard Ahmed et al. [19] as the only direct precedent.

Outside real-time systems, heterogeneous and GPU frameworks exist for graph processing, e.g., Totem [21] and Gunrock [22], and for general task scheduling, e.g., StarPU [23]. Unlike SAGA, all assume uniform per-element work or statically partitionable workloads. Closest are parallel explicit-state model checkers: GPUexplore [24] on the GPU, multi-core SPIN [25] and DiVinE [26] across CPU cores and cluster nodes. All confront state explosion, but their states are concrete configurations deduplicated by equality; none must produce the SIMT-uniform work of $\$V$ or merge interval-valued states under a non-transitive compatibility relation ($\$VI$).

Finally, exact tests for *sporadic* task systems [27–30] cover an infinite family of arrival sequences rather than a finite job set. They are complementary as they prune how many states must be visited; SAGA raises how fast states are visited.

IX. CONCLUSION AND FUTURE WORK

This paper introduced SAGA, a CPU–GPU framework for exploration-merging schedulability analyses, instantiated for the SAG family. Although irregular state-space exploration with order-dependent merging is notoriously hostile to data-parallel hardware, SAGA overcomes these challenges through wave-based streaming, warp-cooperative expansion, a sort-based merge decomposition, and extensible pre-filtering, delivering speedups of two to three orders of magnitude over state-of-the-art CPU tools. Crucially, SAGA does not replace the algorithmic refinements that have shaped the SAG family. Because its expansion is equivalence-preserving and its merge is sound, both past and future optimizations compose directly within our framework.

By turning hours-long analyses into seconds, SAGA shifts accurate schedulability analysis from a last-resort certification step to a routine, design-time utility. Moreover, this paradigm extends well beyond real-time scheduling: timed-automata engines such as UPPAAL and TChecker face similar sequential exploration bottlenecks and can leverage the same abstract operators. Ultimately, we view SAGA as a foundational step toward algorithm–hardware co-design in real-time verification, and an invitation for deeper collaboration between the real-time systems and HPC communities.

ACKNOWLEDGMENT

We would like to thank the anonymous reviewers for their insightful comments on this paper. This work was supported in part by the Institute for Computing, Information and Cognitive Systems (ICICS) at the University of British Columbia (UBC). ChatGPT (OpenAI) and Claude were used as language-editing aids, to improve the grammar, phrasing, and readability of author-prepared draft text throughout the manuscript. Claude was additionally used for general coding assistance and to generate the TikZ sources for Figs. 3 and 5.

REFERENCES

- [1] M. Nasri and B. B. Brandenburg, “An exact and sustainable analysis of non-preemptive scheduling,” in *2017 IEEE Real-Time Systems Symposium (RTSS)*, IEEE, 2017, pp. 12–23.
- [2] M. Nasri, G. Nelissen, and B. B. Brandenburg, “A response-time analysis for non-preemptive job sets under global scheduling,” in *30th Euromicro Conference on Real-Time Systems (ECRTS)*, ser. LIPIcs, vol. 106, Schloss Dagstuhl, 2018, 9:1–9:23. doi: 10.4230/LIPIcs.ECRTS.2018.9.
- [3] G. Nelissen, J. Marcè i Igual, and M. Nasri, “Response-time analysis for non-preemptive periodic moldable gang tasks,” in *34th Euromicro Conference on Real-Time Systems (ECRTS)*, ser. LIPIcs, vol. 231, Schloss Dagstuhl, 2022, 12:1–12:22. doi: 10.4230/LIPIcs.ECRTS.2022.12.
- [4] M. Nasri, G. Nelissen, and B. B. Brandenburg, “Response-time analysis of limited-preemptive parallel DAG tasks under global scheduling,” in *31st Euromicro Conference on Real-Time Systems (ECRTS)*, ser. LIPIcs, vol. 133, Schloss Dagstuhl, 2019, 21:1–21:23. doi: 10.4230/LIPIcs.ECRTS.2019.21.
- [5] S. Srinivasan, M. Günzel, and G. Nelissen, “Response-time analysis for limited-preemptive self-suspending and event-driven delay-induced tasks,” in *2024 IEEE Real-Time Systems Symposium (RTSS)*, IEEE, 2024, pp. 29–42.
- [6] S. Srinivasan, G. Nelissen, R. J. Bril, and N. Meratnia, “Analysis of TSN time-aware shapers using schedule abstraction graphs,” in *36th Euromicro Conference on Real-Time Systems (ECRTS)*, ser. LIPIcs, vol. 298, Schloss Dagstuhl, 2024, 16:1–16:24. doi: 10.4230/LIPIcs.ECRTS.2024.16.
- [7] SAG-org. “nptest: Schedule abstraction graph analysis tool,” Accessed: May 30, 2026. [Online]. Available: https://github.com/SAG-org/schedule_abstraction-main.
- [8] P. Gohari, M. Mikołajczyk, T. van de Klundert, J. Voeten, M. Nasri, and G. Nelissen, “Response-time analysis of conditional DAG tasks under limited-preemptive global scheduling,” in *33rd International Conference on Real-Time Networks and Systems (RTNS)*, 2025.
- [9] M. Foughali, P.-E. Hladik, and A. Zuepke, “Compositional verification of embedded real-time systems,” *Journal of Systems Architecture*, vol. 142, p. 102928, 2023. doi: 10.1016/j.sysarc.2023.102928.
- [10] J. Hansen, S. Srinivasan, G. Nelissen, and K. G. Larsen, “Exact schedulability analysis for limited-preemptive parallel applications using timed automata in UPPAAL,” in *2025 Design, Automation & Test in Europe Conference (DATE)*, IEEE, 2025, pp. 1–7. doi: 10.23919/DATE64628.2025.10992936.
- [11] S. Ranjha, P. Gohari, G. Nelissen, and M. Nasri, “Partial-order reduction in reachability-based response-time analyses of limited-preemptive DAG tasks,” *Real-Time Systems*, vol. 59, no. 2, pp. 201–255, 2023.
- [12] P. Gohari, G. Nelissen, J. Voeten, and M. Nasri, “Leveraging parallelism in global scheduling to improve state space exploration in the SAG framework,” in *Proceedings of the 32nd International Conference on Real-Time Networks and Systems*, 2024, pp. 70–81.
- [13] P. Ekberg and W. Yi, “Uniprocessor feasibility of sporadic tasks remains coNP-complete under bounded utilization,” in *2015 IEEE Real-Time Systems Symposium*, IEEE, 2015, pp. 87–95.
- [14] V. Bonifaci, A. Marchetti-Spaccamela, S. Stiller, and A. Wiese, “Feasibility analysis in the sporadic DAG task model,” in *25th Euromicro Conference on Real-Time Systems (ECRTS)*, IEEE, 2013, pp. 225–233. doi: 10.1109/ECRTS.2013.32.
- [15] E. Lindholm, J. Nickolls, S. Oberman, and J. Montrym, “NVIDIA Tesla: A unified graphics and computing architecture,” *IEEE Micro*, vol. 28, no. 2, pp. 39–55, 2008. doi: 10.1109/MM.2008.31.
- [16] C. P. Pfleeger, “State reduction in incompletely specified finite-state machines,” *IEEE Transactions on Computers*, vol. C-22, no. 12, pp. 1099–1102, 1973. doi: 10.1109/T-C.1973.223655.
- [17] R. M. Karp, “Reducibility among combinatorial problems,” in *Complexity of Computer Computations*, R. E. Miller and J. W. Thatcher, Eds., New York: Plenum Press, 1972, pp. 85–103.
- [18] S. Bereg, S. Cabello, J. M. Díaz-Báñez, P. Pérez-Lantero, C. Seara, and I. Ventura, “The class cover problem with boxes,” *Computational Geometry*, vol. 45, no. 7, pp. 324–334, 2012. doi: 10.1016/j.comgeo.2012.02.002.
- [19] M. Ahmed, S. Rampersaud, N. Fisher, D. Grosu, and L. Schwiebert, “GPU-based parallel EDF-schedulability analysis of multi-modal real-time systems,” in *Proceedings of the 15th IEEE International Conference on High Performance Computing and Communications (HPCC) and the 11th International Conference on Embedded and Ubiquitous Computing (EUC)*, 2013, pp. 254–263. doi: 10.1109/HPCC.and.EUC.2013.45.
- [20] J. Goossens and S. K. Baruah, “Multiprocessor preprocessing algorithms for uniprocessor on-line scheduling,” in *21st International Conference on Distributed Computing Systems (ICDCS)*, IEEE, 2001, pp. 219–226.

- [21] A. Gharaibeh, L. B. Costa, E. Santos-Neto, and M. Ripeanu, “A yoke of oxen and a thousand chickens for heavy lifting graph processing,” in *Proceedings of the 21st International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2012, pp. 345–354.
- [22] Y. Wang, A. Davidson, Y. Pan, Y. Wu, A. Riffel, and J. D. Owens, “Gunrock: A high-performance graph processing library on the GPU,” in *Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2016, pp. 1–12.
- [23] C. Augonnet, S. Thibault, R. Namyst, and P.-A. Wacrenier, “StarPU: A unified platform for task scheduling on heterogeneous multicore architectures,” *Concurrency and Computation: Practice and Experience*, vol. 23, no. 2, pp. 187–198, 2011.
- [24] A. Wijs and D. Bošnački, “GPUexplore: Many-core on-the-fly state space exploration using GPUs,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, Springer, 2014, pp. 233–247.
- [25] G. J. Holzmann and D. Bošnački, “The design of a multicore extension of the SPIN model checker,” *IEEE Transactions on Software Engineering*, vol. 33, no. 10, pp. 659–674, 2007.
- [26] J. Barnat, L. Brim, I. Černá, P. Moravec, P. Ročkai, and P. Šimeček, “DiVinE – a tool for distributed verification,” in *Computer Aided Verification (CAV)*, Springer, 2006, pp. 278–281.
- [27] T. P. Baker and M. Cirinei, “Brute-force determination of multiprocessor schedulability for sets of sporadic hard-deadline tasks,” in *Principles of Distributed Systems (OPODIS)*, Springer, 2007, pp. 62–75.
- [28] G. Geeraerts, J. Goossens, and M. Lindström, “Multi-processor schedulability of arbitrary-deadline sporadic tasks: Complexity and antichain algorithm,” *Real-Time Systems*, vol. 49, no. 2, pp. 171–218, 2013.
- [29] A. Burmyakov, E. Bini, and C.-G. Lee, “Towards a tractable exact test for global multiprocessor fixed priority scheduling,” *IEEE Transactions on Computers*, vol. 71, no. 11, pp. 2955–2967, 2022. doi: 10.1109/TC.2022.3142540.
- [30] S. Picard, A. Paolillo, G. Geeraerts, and J. Goossens, “Exact schedulability test for sporadic mixed-criticality real-time systems using antichains and oracles,” in *Proceedings of the 32nd International Conference on Real-Time Networks and Systems (RTNS)*, ACM, 2024, pp. 302–314. doi: 10.1145/3696355.3699702.